

A_DRAM — DRAM 셀·동작·타이밍

🔴 1T1C 셀에 전하로 데이터를 저장하고 sense amp 로 미세전압을 감지·증폭하는 휘발성 메모리, 누설 때문에 주기적 refresh 필요 · 📇 P2 · 🧠 tRCD·tRP·CL 같은 타이밍은 컨트롤러 FSM·STA 제약 그 자체 — 디지털 설계자의 "셋업/홀드 in DRAM 프로토콜"

핵심 개념

- **1T1C 셀**: 1 access transistor + 1 storage capacitor. WL=transistor gate, BL=전하 통로. 셀 캡 ~수~수십 fF, 읽기는 destructive(파괴적) → 반드시 write-back.
- **셀 면적 $6F^2 \rightarrow 4F^2$** : F=최소 피처. $6F^2(2F \times 3F)$ 가 표준, $4F^2(2F \times 2F) + VG(\text{Vertical Gate/수직채널 VCT})$ 로 다이 ~30%↓ (SK하이닉스 VLSI 2025 로드맵). 그 다음은 **3D DRAM**(셀 수직 적층).
- **노드 1a/1b/1c**: 10nm-class 4/5/6세대(1a≈14, 1b≈12, 1c≈11nm급). 1c=2025~26 양산, EUV 5~6 layer, 1c DDR5 8Gbps(+14% 속도/-9% 전력).
- **Sense amp 동작**: ① BL pair를 **VDD/2 precharge** → ② WL on → ③ 셀 캡과 BL 기생캡 **charge sharing**으로 ±수십 mV 미세 차이 → ④ cross-coupled latch가 **차동 증폭**해 full-rail → 동시에 셀 **restore**(재기록).
- **Precharge(tRP)**: 다음 access 전 BL을 다시 VDD/2로 equalize, row close.
- **Refresh / retention**: 캡 전하가 누설로 사라짐 → 주기적 read+rewrite. **retention 표준 64ms**(고온 32ms), $tREFI \approx 7.8\mu s$ (8192 row/64ms). 미세화로 저장전하↓ → retention이 핵심 스케일링 난제. DDR5: same-bank refresh + on-die ECC로 보강.
- **Bank / Bank group**: 독립 array=bank, 각 bank가 row 1개 open 가능 → 동작 overlap으로 latency 은닉. DDR5 16Gb=8 BG × 4 bank = 32 bank. 같은 BG=tCCD_L(느림), 다른 BG=tCCD_S(빠름).
- **타이밍(같은 bank 기준)**:
 - **tRCD (RAS→CAS)**: ACT(row open)→READ/WRITE 최소지연. WL+sense amp 안정 시간.
 - **tRP (Row Precharge)**: PRE→다음 ACT. row close/BL precharge.
 - **tRAS (Row Active)**: ACT→PRE 최소. 셀 restore 충분 시간 보장.
 - **tRC (Row Cycle)**: ACT→다음 ACT = **tRC = tRAS + tRP**.
 - **CL=tCAS (CAS Latency)**: READ→첫 data out, clock cycle 단위.
 - **tWR (Write Recovery)**: 마지막 write data→PRE. 셀에 다 써질 시간.
- **DDR5 신기능**: **on-die ECC**(미세노드 셀 불량 보정), **DFE 4-tap**(고속 ISI 보상), **4800→8800 MT/s**(JESD79-5C, 2024.04), DIMM당 독립 32b 2-subchannel, on-DIMM PMIC.
- **Peri(주변회로)**: cell array 밖의 모든 로직 — row/col decoder, WL driver, sense amp, charge pump, command/address, DQ I/O. **DRAM 안의 "디지털 설계" 영역**. Cell-over-Peri(COP)로 면적 절감.
- **종류**: DDR(서버/PC, DIMM) / LPDDR(모바일 저전력, point-to-point) / GDDR(GPU, per-pin 초고속 ~24Gbps) / HBM(3D 적층, 초광대역).

예상 면접 Q&A

Q1. DRAM 1T1C 셀이 데이터를 저장·읽는 과정을 설명하라.

A. 셀은 access transistor 1개와 storage capacitor 1개로 구성됩니다. 쓰기는 WL로 transistor를 켜고 BL의 전압(0/VDD)을 캡에 충·방전합니다. 읽기는 먼저 BL pair를 VDD/2로 precharge한 뒤 WL을 켜면, 셀 캡과 BL 기생캡 사이 **charge sharing**으로 BL에 $\pm$ 수십 mV의 미세 차이가 생깁니다. sense amp(cross-coupled latch)가 이 차동 신호를 full-rail로 증폭해 0/1을 판정합니다. 읽기는 셀 전하를 흘뜨리는 **파괴적 동작**이라, 같은 증폭으로 셀에 **다시 써넣는(restore)** 과정이 항상 동반됩니다. - L, 꼬리질문: 왜 VDD/2 precharge인가? → charge sharing 시 0/1 어느 쪽이든 대칭적 신호 마진 확보 + 차동 감지에 최적, 전력도 절반. - 🧠 설계 브릿지: sense amp는 아날로그처럼 보이지만 그 앞단 timing(precharge→WL→enable)은 컨트롤러 FSM이 만드는 **순서 제약**. 디지털 설계자는 이 sequencing을 STA 제약으로 본다.

Q2. tRCD, tRP, tRAS, tRC, CL, tWR을 정의하고 관계를 설명하라.

A. 모두 같은 bank 기준입니다. **tRCD**=ACTIVATE(row open)에서 READ/WRITE 가능까지(WL·sense amp 안정), **CL(tCAS)**=READ 명령에서 첫 데이터 출력까지(clock 단위), **tRP**=PRECHARGE에서 다음 ACTIVATE까지(row close/BL precharge), **tRAS**=ACTIVATE에서 PRECHARGE까지 최소(셀 restore 보장), **tWR**=마지막 write 데이터에서 PRECHARGE까지(셀에 다 써질 시간). 핵심 관계는 **tRC = tRAS + tRP**(한 row를 열고 닫는 전체 cycle), 그리고 random read latency $\approx$ **tRCD + CL**입니다. - L, 꼬리질문: tRAS를 너무 짧게 잡으면? → restore가 덜 끝나 셀 전하가 부족 → retention/데이터 깨짐. 그래서 하한이 있다. - 🧠 설계 브릿지: 이 파라미터들은 메모리 컨트롤러가 지키는 **타이밍 제약 집합** = 디지털 설계의 setup/hold·multicycle path와 동일한 사고방식. 어기면 functional fail.

Q3. Refresh가 왜 필요하고 주기는 어떻게 정해지나?

A. storage capacitor의 전하는 transistor 누설·접합 누설로 시간이 지나면 사라집니다. 데이터가 깨지기 전에 주기적으로 읽어서 다시 써넣어야 하는데 이것이 refresh입니다. 셀이 데이터를 유지하는 시간이 **retention time**이고 표준 spec은 **64ms**(85°C 초과 고온에서는 절반인 32ms)입니다. 전체 row(예 8192개)를 64ms 안에 한 번씩 돌려야 하므로 평균 refresh 간격 **tREFI $\approx$ 7.8 μ s**마다 auto-refresh를 발행합니다. 미세화로 셀이 작아질수록 저장 전하가 줄어 retention이 어려워지고, 이게 DRAM 스케일링의 근본 난제입니다. - L, 꼬리질문: refresh의 단점은? → refresh 중 해당 bank 접근 불가 → 가용 대역폭·전력 손실(특히 고용량·고온). DDR5는 same-bank refresh로 일부 완화. - 🧠 설계 브릿지: refresh는 백그라운드 housekeeping FSM. 정상 read/write와 충돌 안 나게 스케줄링하는 것은 컨트롤러 arbitration 설계 문제.

Q4. Bank와 bank group이 왜 필요한가?

A. DRAM을 여러 독립 array인 **bank**로 나누면 각 bank가 동시에 서로 다른 row를 open한 상태를 가질 수 있어, 한 bank의 activate/precharge 지연을 다른 bank 접근으로 **숨겨(overlap)** 대역폭을 끌어올립니다. DDR4/5는 bank를 다시 **bank group(BG)**으로 묶는데, 같은 BG 내 연속 접근(tCCD_L)은 공유 회로 때문에 느리고, 다른 BG 접근(tCCD_S)은 빠릅니다. 그래서 컨트롤러는 연속 요청을 일부러 다른 BG로 분산해 burst를 촘촘히 채웁니다. DDR5 16Gb은 8 BG $\times$ 4 bank = 32 bank입니다. - L, 꼬리질문: bank를 무한정 늘리면? → peri(decoder·sense amp) 오버헤드와 면적 증가, 제어 복잡도 $\uparrow$. trade-off. - 🧠 설계 브릿지: bank 병렬성은 디지털 설계의 pipelining/리소스 분할과 같은 개념 — 자원을 나눠 동시성을 만들어 throughput을 높인다.

Q5. DDR / LPDDR / GDDR / HBM의 차이는?

A. 같은 DRAM 셀을 쓰되 **타깃과 I/O 전략**이 다릅니다. **DDR**은 서버/PC용 범용, DIMM 모듈, 균형형(DDR5 4800~8800 MT/s). **LPDDR**은 모바일 저전력으로 저전압·deep power-down·point-to-point(패키지 실장). **GDDR**은 GPU용으로 per-pin 속도를 극단으로(GDDR6 16~24Gbps) 올린 대신 전력이 큼. **HBM**은 DRAM을 TSV로 수직 적층해 1024/2048-bit 초광대역 I/O를 moderate 속도로 굴러, **bit당 에너지 최저 + 대역폭 최고 + 면적 최소화**를 달성합니다(2.5D 인터포저). 즉 DDR=용량·범용, LPDDR=전력, GDDR=per-pin 속도, HBM=총대역폭·집적입니다. - L, 꼬리질문: AI 가속기가 GDDR 대신 HBM을 쓰는 이유? → 같은 대역폭을 훨씬 낮은 전력·작은 면적으로, 그리고 한 패키지에 수십~수백 GB/s 핀이 아닌 TB/s급을 집적 가능. - 🧠 설계 브릿지: I/O 폭 vs per-pin 속도 trade-off는 SerDes·버스 폭 설계와 동일. 폭을 늘리면 SI는 쉬워지나 면적·핀이 늘고, 속도를 올리면 DFE 같은 등화가 필요.

Q6. DDR5에서 새로 들어온 기능과 그 이유는?

A. 대표적으로 **on-die ECC**, **DFE**, 그리고 속도 확장(**4800→8800 MT/s**, JESD79-5C 2024)입니다. on-die ECC는 미세 노드에서 늘어나는 single-bit 셀 불량률을 칩 내부에서 보정해 수율·신뢰성을 지킵니다. DFE(4-tap decision feedback equalizer)는 고속에서 심해지는 ISI(심볼간 간섭)를 receiver에서 보상합니다. 구조적으로는 DIMM을 독립 32-bit sub-channel 2개로 쪼개 효율을 높이고, on-DIMM PMIC로 전원을 모듈에서 직접 관리합니다. - L, 꼬리질문: on-die ECC가 있으면 시스템 ECC는 불필요한가? → 아니다. on-die ECC는 셀 단위 보정용이고, 시스템 ECC(rank 단위)는 링크·소자 전체 오류 검출/정정으로 역할이 다르다. - 🧠 설계 브릿지: DFE·on-die ECC는 SI/신뢰성 회로 — 디지털 설계자가 STA에서 보는 jitter/eye margin, 그리고 ECC 인코더/디코더 RTL과 직접 연결.

Q7. Peri(주변회로)란 무엇이고 왜 설계자에게 중요한가?

A. Peri는 cell array를 제외한 모든 회로 — row/column decoder, WL driver, sense amp, charge pump(승압), command/address 디코딩, DQ I/O, 타이밍 제어 로직입니다. 셀은 소자·공정의 영역이지만 **peri는 사실상 DRAM 안의 디지털·혼성신호 설계 영역**이라 제 RTL/STA/CDC 경험이 직접 닿는 곳입니다. 최근에는 면적 절감을 위해 peri를 셀 아래로 넣는 **Cell-over-Peri(COP)** 구조도 씁니다. - L, 꼬리질문: peri가 미세화에서 받는 압박은? → 셀은 4F²/3D로 줄이는데 peri 로직·sense amp는 그만큼 안 줄어 면적 비중이 커짐 → COP·고집적 peri 설계가 중요. - 🧠 설계 브릿지: peri의 command/address path, refresh FSM, ECC, 테스트 로직은 전형적 디지털 블록 — 제 sign-off(Lint/CDC/STA) 방법론이 그대로 적용된다.

Q8. 6F²→4F²와 3D DRAM 추세를 설명하라.

A. 평면 셀은 6F²(2F×3F)가 오래 표준이었는데, transistor를 수직으로 세운 **VCT(vertical channel transistor)+VG**로 셀을 **4F²(2F×2F)**까지 줄이면 같은 노드에서 다이들 ~30% 줄일 수 있습니다(SK하이닉스 VLSI 2025 로드맵). 전류 경로가 캡→수직채널→매립 BL로 짧아지는 이점도 있습니다. 평면 미세화가 한계에 닿으면 NAND처럼 셀을 수직 적층하는 **3D DRAM**으로 가는데, DRAM은 캡과 access tr을 3D로 쌓는 난도가 NAND보다 훨씬 높습니다. - L, 꼬리질문: DRAM 미세화가 NAND보다 어려운 이유? → 충분한 셀 capacitance를 유지해야 sensing/retention이 되는데, 면적을 줄이면 캡을 길쭉하게(고종횡비) 만들어야 해 제조가 극히 까다롭다. - 🧠 설계 브릿지: 4F²/3D로 셀이 작아질수록 신호 마진이 줄어 on-die ECC·정교한 sense amp 타이밍이 더 중요 — 소자 미세화가 곧 회로/검증 난도 상승.

30초 암기 요약

- 1T1C: WL=gate, BL=전하통로, 읽기는 파괴적→restore 필수. 셀 $6F^2 \rightarrow 4F^2(VG/VCT) \sim 30\% \downarrow$, 다음은 3D DRAM.
- 노드 1a ≈ 14 / 1b ≈ 12 / 1c $\approx 11\text{nm}$ 급(2025~26 양산, EUV), 1c DDR5 8Gbps.
- Sense amp: BL VDD/2 precharge $\rightarrow$ WL on $\rightarrow$ charge sharing($\pm$ 수십mV) $\rightarrow$ 차동 증폭 $\rightarrow$ restore.
- **retention 64ms(고온 32ms)**, **tREFI $\approx 7.8\mu\text{s}$** . 누설 때문에 refresh 필수, 미세화=retention 난제.
- **tRC = tRAS + tRP**, random latency $\approx$ tRCD + CL. tWR=write $\rightarrow$ precharge.
- DDR5 16Gb = **8 BG $\times$ 4 bank = 32 bank**, same-BG=tCCD_L 느낌 / diff-BG=tCCD_S.
- DDR5 신기능: **on-die ECC + DFE(4-tap) + 4800~8800 MT/s**, 32b sub-channel $\times 2$.
- DDR=범용 / LPDDR=저전력 / GDDR=per-pin 초고속 / HBM=초광대역. Peri=DRAM 속 디지털 설계 영역.